TAKE-HOME ASSIGNMENT: FULLSTACK DEVELOPER, ENTERPRISE AI PLATFORMS

Assignment ID: Fullstack-ClaudeGemini-UsageConnector-v1

The assignment

Meridian Life Sciences (fictional) runs an enterprise AI assistant for its staff on Claude and Gemini. You are on the platform team. The platform owners want to see how much the assistant is being used, by whom and at what cost, and to control which groups may use which capabilities. Then they want that same information available inside the assistant itself, so that a team lead can ask "how much did my team use last month?" without leaving it, and see only their own team.

The starter kit that comes with this brief has usage exports from both platforms in their native shapes, and a user and group directory that stands in for the identity provider. Treat these as the systems of record.

Two parts. Part A is the core; Part B builds on it.

We care as much about how you build it as what you build.

Part A: Usage and admin service

Build the service the platform owners will use.

- **Ingest both exports into one model.** The Claude export is per-user daily token usage and cost from the Claude Enterprise Analytics API, broken out by product and model, with cost as decimal strings in cents; the Gemini export is Google Cloud billing rows for Vertex AI token SKUs, per project, with cost in dollars and attribution through labels. Granularity, units, identifiers and timestamp formats differ. Attribute usage to users and groups through the directory.
- **Expose an API and a minimal admin page:** usage and cost by platform, by group and by week, and per-group capability controls (enable or disable a capability for a group), persisted, with authorisation on who may change them. Identity is simulated: the caller is a user from the directory, passed however you like (a header, a query parameter, a user switcher on the page), and their role decides what they may see and change.
- **Test the mapping.** Backend first; the UI can be plain.



Part B: Expose it to the assistant, scoped

Make the service available inside the assistant, the way the platform team would ship any connector.

- **Build the connector** as an MCP server over your Part A service, with four tools: `usage_summary(group?, period)`, `top_users(group?, period, limit?)`, `capability_status(group?)` and `set_capability(group, capability, enabled)`.
- **Scope the access.** Identity is simulated: the server starts as a given user and takes that user's groups and role from the directory. Team leads see their own group only; platform admins see every group; `set_capability` needs the admin scope. Enforce it in the server, not in the prompt, and test it: a team lead asking for another group's usage gets a structured denial and no data. The server also honours the per-group controls from Part A before answering.
- **Register and distribute it** on your platform. Claude: register it in Claude Desktop as a local MCP server or as a custom connector, and package it as a Claude plugin. Gemini: register it in Gemini CLI and package it as a Gemini CLI extension. Show it working with a short recording or transcript: an allowed read, a denied cross-group read, a denied write.
- **Automate the provisioning.** Three environments, dev, staging and prod, with different grants (the starter kit says which). One command renders the permissions manifest and the host registration config per environment and validates it; CI runs your tests and that validation. Terraform with local providers is welcome; a clean script is fine.
- **Close the loop.** A short note to the team leads who asked for this, in their language: what they can now ask, what they cannot and why, how to ask for more.

What to submit

1. **GitHub repo** with your code. Python and/or TypeScript; we would like to see both if it falls out naturally, say the service in one and the connector in the other. Do not contort the design for it.
2. **README.md** with: quick setup instructions; architecture overview with a diagram (a simple one is fine, but we do need one); your mapping decisions, from two exports to one model and from rows to people and groups; how the permission model works and where it is enforced; which platform you built for and exactly what you ran it on; key technical decisions and why; engineering standards you followed (and some you skipped); how you used AI tools in your development process; what you would do differently with more time; and a section titled *In the real enterprise version*: what changes when the identity provider, the organisation-level connector





registry, the platforms' real usage APIs and real infrastructure exist. We need your thoughts, not an LLM's direct output.

3. **The recording or transcript** from Part B, and the note to the team leads.

What we're looking for

- **Backend craft.** Honest mapping of two messy exports into one model, clean structure others can read, and tests that would catch a regression.
- **A permission model that holds.** Enforced in code, minimal by default, readable by a platform owner, and impossible to talk around. This is the part we test hardest.
- **Platform depth.** Registration and packaging follow the platform's documented conventions; the README shows you understand how capabilities are scoped and distributed there, and where usage data comes from on that platform.
- **Provisioning discipline.** One command renders all three environments, the configuration is validated, CI is green, and nothing is hand-edited.
- **Writing.** The README and the note to the team leads are a writing sample; this role writes for requesters and platform owners every week.

Tips & notes

- Start simple, then enhance. A working subset with clear notes beats a sprawling repository where nothing quite runs. We will not judge you on every edge case, but do acknowledge them in your README.
- Use AI coding tools freely. Tell us how you used them and what you overruled.
- Not enough time to do everything? Document what you would add next.
- The export shapes, the directory, the roles and the environment differences are all described in the starter kit's README.

